



Step 1: Look at the raw table

Attributes:
projectNumber projectName employeeNumber employeeName jobType hourlyRate hoursWorked

ProjectAssignment (1NF)
PK projectNumber
FK employeeNumber
projectName
employeeName
jobType
hourlyRate
hoursWorked

Step 4: Identify dependencies

1. **projectNumber** → **projectName**
• Atomic values! (every cell has one value, no repeating groups). In databases, this means each column contains a single, distinct piece of information, such as a name or date, not a list or set of values.
2. **employeeNumber** → **employeeName, jobType**
• Employee number determines their name and job type.
3. **jobType** → **hourlyRate**
• If you know the job type, you know the hourly rate.
4. **(projectNumber, employeeNumber)** → **hoursWorked**
• This is the key → hoursWorked is specific to that combination

A database is in **First Normal Form (1NF)** if each of its columns contains only atomic values (indivisible, single values) and there are no repeating groups of columns or rows.



Book Example 1

Book (1NF)
PK bookTitle
PK edition
authorNumber
lastName
publisher
royalty

book	authorNumber	lastName	publisher	royalty	edition
bookTitle					
Introduction to programming	453 Williams	Collins	25	4	
Introduction to programming	453 Williams	Collins	29	5	
SQL Explained	225 Smith	Collins	34	4	
Normalization	255 Brown	Pearson	23	2	
Database Theory	434 Turner	Wiley	22	3	
Object Oriented Programming	453 Williams	Pearson	14	5	
Oracle for Beginners	223 Baker	Collins	29	3	
Primary keys	225 Smith	Cengage	32	1	
What is a table	523 Jones	Pearson	23	5	
Relations Explained	244 Robinson	Persea	42	3	
Python Programming	523 Jones	Hypertion	14	2	

- Step 3: Find candidate keys
- Composite key (bookTitle, edition) uniquely identifies a row [since books can have multiple editions].
 - authorNumber is not unique (one author writes many books).
 - lastName isn't unique (different authors could share surnames).
- So, (bookTitle, edition) → primary key.

Text

- Step 2: Define important terms
- **Primary Key (PK):** A unique identifier for a row. Example: (bookTitle, edition) could work, since a book may have multiple editions.
 - **Candidate Key:** Any attribute (or combination) that could uniquely identify a row. Example: (bookTitle, edition) is one candidate; (authorNumber, bookTitle, edition) could also be—but that introduces redundancy.
 - **Functional Dependency (FD):** X → Y means: If you know X, you can determine Y.
 - Example: bookTitle → authorNumber, publisher (a title has one author and one publisher).
 - **Partial Dependency:** When a non-key attribute depends on part of a composite key.
 - Example: (bookTitle, edition) → royalty, but publisher depends only on bookTitle, not on edition.
 - **Transitive Dependency:** When a non-key depends on another non-key.
 - Example: authorNumber → lastName. Then bookTitle → authorNumber → lastName.

• Primary Key: (bookTitle, edition).

- Current PK = (bookTitle, edition).
- Problem:
 - bookTitle → authorNumber, lastName, publisher (depends only on bookTitle, not edition).
 - royalty depends on the full key (bookTitle + edition).
- **Fix:** Split tables:
 - **Book(bookTitle, authorNumber, publisher)**
 - **Book_Edition(bookTitle, edition, royalty)**
 - **Author(authorNumber, lastName)**This removes partial dependency because now publisher and author info depend only on bookTitle, not on (bookTitle + edition).

- Step 4: Check Normal Forms
- 1NF (Eliminate repeating groups)**
- Already satisfied: each cell has atomic values (no lists).
 - But there are **redundancies**: publishers, authors, royalties repeated.

Does this table have a primary key? If so, what is it?

Is this table in a Normal Form? If so, what Normal Form is it in?

(projectNumber, employeeNumber)

Why?

- **projectNumber alone** is not unique (many employees work on the same project).
- **employeeNumber alone** is not unique (an employee can work on multiple projects).
- But together, (projectNumber + employeeNumber) uniquely identifies one assignment of an employee to a project. So:
 - That's the **composite primary key**.
 - Every other attribute (projectName, employeeName, jobType, hourlyRate, hoursWorked) is **functionally dependent** on that composite key — and this is where normalization kicks in, because some attributes only depend on part of the key (partial dependencies) or on non-key attributes (transitive dependencies).

Step 5: What normal form is it in?

1NF (First Normal Form):
• Atomic values! (every cell has one value, no repeating groups). In databases, this means each column contains a single, distinct piece of information, such as a name or date, not a list or set of values.
• Primary key defined! (composite key works).
So it's at least in 1NF.

2NF (Second Normal Form):

- Rule: No partial dependency on part of a composite key.
- Problem: employeeNumber → employeeName, jobType (depends only on part of the key).
- So not in 2NF.
- **3NF (Third Normal Form):**
 - Rule: No transitive dependencies.
 - Problem: jobType → hourlyRate (that's a transitive dependency via employeeNumber).
 - So not in 3NF either.

2NF (Remove partial dependencies)

• Primary Key: (bookTitle, edition).

- Current PK = (bookTitle, edition).
- Problem:
 - bookTitle → authorNumber, lastName, publisher (depends only on bookTitle, not edition).
 - royalty depends on the full key (bookTitle + edition).
- **Fix:** Split tables:
 - **Book(bookTitle, authorNumber, publisher)**
 - **Book_Edition(bookTitle, edition, royalty)**
 - **Author(authorNumber, lastName)**This removes partial dependency because now publisher and author info depend only on bookTitle, not on (bookTitle + edition).

- Step 4: Check Normal Forms
- 1NF (Eliminate repeating groups)**
- Already satisfied: each cell has atomic values (no lists).
 - But there are **redundancies**: publishers, authors, royalties repeated.



Assignment table just stores FKs (projectNumber, employeeNumber) and hoursWorked.

Project Example 2

projectNumber	employeeNumber	hoursWorked
35	103	32
35	101	23
35	105	43
38	106	23
38	102	12
22	105	54
22	106	12
22	113	30
22	111	21
22	106	21
25	107	14
25	115	15
28	101	13
25	114	23
25	113	21